

2주차

📅 날짜	2026년 9월 15일
≡ 텍스트	파머완 2장, 3장

2장 사이킷런으로 시작하는 머신러닝

1. 사이킷런 소개와 특징

사이킷런(scikit-learn)

- 가장 많이 사용되는 대표적인 파이썬 ML 라이브러리
- 쉽고 가장 파이썬스러운 API 제공
- 다양한 알고리즘, 편리한 프레임워크&API 제공
- 오랜기간 검증됨, 많은 환경에서 사용되는 성숙한 라이브러리

2. 첫 번째 머신러닝 만들어 보기 - 붓꽃 품종 예측하기

붓꽃 데이터세트

- 붓꽃 데이터 세트로 붓꽃 품종 분류(Classification)
- 피쳐(Feature) : 꽃잎의 길이&너비, 꽃받침 길이&너비
- 레이블(Label) : 꽃 품종 (Setosa, Vesicolor, Virginica)



지도학습(Supervised Learning)이란?

레이블 데이터로 모델 학습 → 미지의 레이블 예측
= 명확한 정답 주어진 데이터 먼저 학습 → 미지의 정답 예측

레이블(Label) 데이터 : 학습을 위한 다양한 피쳐와 분류 결정값

주어진 데이터 세트 : **학습 데이터 세트**

머신러닝 모델 예측 성능 평가 위한 데이터 세트 : **테스트 데이터 세트**

분류(Classification)는 대표적인 지도학습!

사이킷런에서 사용할 모듈 임포트

- 사이킷런 패키지 내의 모듈명 `sklearn` 로 시작하는 명명규칙

<code>sklearn.dataset</code> 내 모듈	사이킷런에서 자체적으로 제공하는 데이터 세트 생성하는 모듈 모임	<code>from sklearn.datasets import load_iris</code>	<code>load_iris()</code> 함수	붓꽃 데이터 세트 생성
<code>sklearn.tree</code> 내 모듈	트리 기반 ML 알고리즘 구현한 클래스 모임	<code>from sklearn.tree import DecisionTreeClassifier</code>	<code>DecisionTreeClassifier</code>	ML 알고리즘인 의사 결정 트리 (Decision Tree) 알고리즘 구현한 클래스

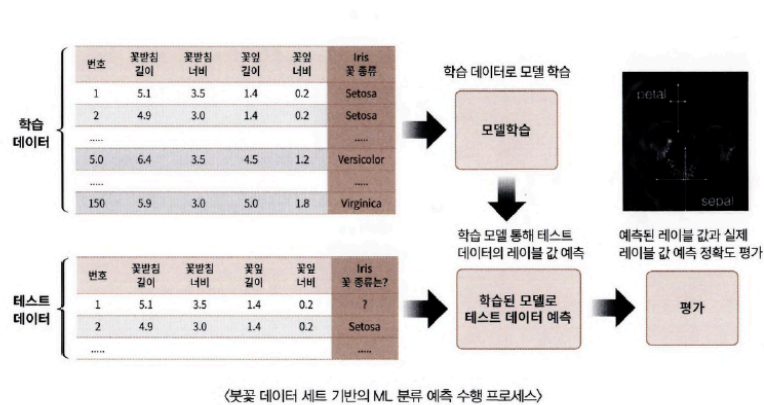
<code>sklearn.model_selection</code>	학습 데이터, 검증 데이터, 예측 데이터로 데이터 분리 or 최적의 하이퍼 파라미터 로 평가 위한 다양한 모듈 모임	<code>from sklearn.model_selection import train_test_split</code>	<code>train_test_split()</code> 함수	학습 데이터와 테스트 데이터로 분리 <code>test_size</code> 파라미터 : 입력값 비율로 데이터 분할
--------------------------------------	---	---	------------------------------------	---

하이퍼 파라미터

- 머신러닝 알고리즘별로 최적의 학습 위해 직접 입력하는 파라미터들 통칭.
- 하이퍼 파라미터 통해 머신러닝 알고리즘 성능 튜닝 가능

예측 프로세스

- 데이터 세트 분리
- 모델 학습
- 예측 수행
- 평가



3. 사이킷런의 기반 프레임워크 익히기

(1) Estimator 이해 및 fit(), predict() 메서드

- `fit()` 메소드
: ML 모델 학습 위해
- `predict()` 메소드
: 학습된 모델의 예측 위해

지도학습

- `Classifier` 클래스
: 분류 알고리즘 구현한 클래스
- `Regressor` 클래스
: 회귀 알고리즘 구현한 클래스
- `Estimator` 클래스
: `Classifier` + `Regressor` 합친 클래스.
지도학습의 모든 알고리즘 구현한 클래스 통칭.

지도학습 주요 두 축인 분류(Classification)와 회귀(Regression) 다양한 알고리즘 구현한 모든 사이킷런 클래스 `fit()`, `predict()` 만 이용해 간단하게 학습, 예측 결과 반환. `Estimator` 클래스 또한 내부에서 `fit()`, `predict()` 구현

`Estimator` 인자로 받는

<code>cross_val_score()</code>	evaluation 함수	<code>cross_val_score()</code> 함수 내에서 <code>Estimator</code> 의 <code>fit()</code> , <code>predict()</code> 호출해서 평가
<code>GridSearchCV</code>	하이퍼 파라미터 튜닝 지원하는 클래스	<code>GridSearchCV.fit()</code> 함수 내에서 <code>Estimator</code> 의 <code>fit()</code> , <code>predict()</code> 호출해서 하이퍼 파라미터 튜닝 수행

비지도학습

- `fit()`
: 입력 데이터 형태에 맞춰 데이터 변환 위한 사전 구조 맞추는 작업
- `transform()`
: `fit()` 이후 입력데이터 실제 작업(차원 변환, 클러스터링, 피쳐 추출 등) 수행
- `fit_transform()`
: `fit()` + `transform()` 결합. 각각 별도로 호출할 필요 줄여줌

사이킷런에서 비지도학습 구현한 클래스 대부분 `fit()`, `transform()` 적용

(2) 사이킷런 주요 모듈

분류	모듈명	설명
예제 데이터	<code>sklearn.datasets</code>	사이킷런에 내장되어 예제로 제공하는 데이터 세트
피쳐 처리	<code>sklearn.preprocessing</code>	데이터 전처리에 필요한 다양한 가공 기능 제공(문자열을 숫자 형 코드 값으로 인코딩, 정규화, 스케일링 등)
	<code>sklearn.feature_selection</code>	알고리즘에 큰 영향을 미치는 피쳐를 우선순위로 선택 작업 수행하는 다양한 기능 제공

피쳐 처리	<code>sklearn.feature_extraction</code>	텍스트 데이터나 이미지 데이터의 벡터화된 피쳐를 추출하는 데 사용됨. 예를 들어 텍스트 데이터에서 Count Vectorizer나 Tfidf Vectorizer 등을 생성하는 기능 제공. 텍스트 데이터의 피쳐 추출은 <code>sklearn.feature_extraction.text</code> 모듈에, 이미지 데이터의 피쳐 추출은 <code>sklearn.feature_extraction.image</code> 모듈에 지원 API가 있음.
	<code>sklearn.decomposition</code>	차원 축소와 관련된 알고리즘을 지원하는 모듈임. PCA, MFA, Truncated SVD 등을 통해 차원 축소 기능을 수행할 수 있음.
데이터 분리, 검증 & 파라미터 튜닝	<code>sklearn.model_selection</code>	교차 검증을 위한 학습용/테스트용 분리, 그리드 서치(Grid Search)로 최적 파라미터 추출 등의 API 제공
평가	<code>sklearn.metrics</code>	분류, 회귀, 클러스터링, 페어와이즈(Pairwise)에 대한 다양한 성능 측정 방법 제공. Accuracy, Precision, Recall, ROC-AUC, RMSE 등 제공
ML 알고리즘	<code>sklearn.ensemble</code>	앙상블 알고리즘 제공. 랜덤 포레스트, 애디다 부스트, 그래디언트 부스팅 등을 제공
	<code>sklearn.linear_model</code>	주로 선형 회귀, 릿지(Ridge), 라쏘(Lasso) 및 로지스틱 회귀 등 회귀 관련 알고리즘을 지원, 또한 SGD(Stochastic Gradient Descent) 관련 알고리즘도 제공
	<code>sklearn.naive_bayes</code>	나이브 베이즈 알고리즘 제공. 가우시안 NB, 다항 분포 NB 등.
	<code>sklearn.neighbors</code>	최근접 이웃 알고리즘 제공. K-NN 등
	<code>sklearn.svm</code>	서포트 벡터 머신 알고리즘 제공
	<code>sklearn.tree</code>	의사 결정 트리 알고리즘 제공
유틸리티	<code>sklearn.cluster</code>	비지도 클러스터링 알고리즘 제공. (K-평균, 커먼형, DBSCAN 등)
	<code>sklearn.pipeline</code>	피쳐 처리 등의 변환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행할 수 있는 유틸리티 제공

머신러닝 모델 구축 주요 프로세스

- 피쳐 처리(feature processing) : 피쳐의 가공, 변경, 추출 수행
- ML 알고리즘 학습/예측 수행
- 모델 평가

⇒ 단계 반복적으로 수행

(3) 내장된 예제 데이터 세트

- 별도의 외부 웹사이트에서 데이터 세트 내려받을 필요 ❌
- 예제로 활용 가능한 간단하면서도 좋은 데이터 세트 내장
- `datasets` 모듈에 있는 여러 API 호출해 만들

예제 용도 데이터

분류나 회귀 연습용 예제 데이터

API 명	설명
<code>datasets.load_boston()</code>	회귀 용도이며, 미국 보스턴의 집 피쳐들과 가격에 대한 데이터 세트
<code>datasets.load_breast_cancer()</code>	분류 용도이며, 위스콘신 유방암 피쳐들과 양성/음성 레이블 데이터 세트
<code>datasets.load_diabetes()</code>	회귀 용도이며, 당뇨 데이터 세트
<code>datasets.load_digits()</code>	분류 용도이며, 0에서 9까지 숫자의 이미지 픽셀 데이터 세트
<code>datasets.load_iris()</code>	분류 용도이며, 붓꽃에 대한 피쳐를 가진 데이터 세트

fetch 계열 데이터

- 데이터 크기 커서 패키지에 처음부터 저장 ❌
- 인터넷에서 내려받아 홈 디렉터리 아래 서브 디렉터리 `scikit_learn_data` 에 저장 → 추후 불러들임
- 최초 사용 시 인터넷에 연결 안되어있으면 사용 ❌

표본 데이터

분류와 클러스터링을 위한 표본 데이터 생성기

API 명	설명
<code>datasets.make_classifications()</code>	분류를 위한 데이터 세트를 만듭니다. 특히 높은 상관도, 불필요한 속성 등의 노이즈 효과를 위한 데이터를 무작위로 생성해 줍니다.
<code>datasets.make_blobs()</code>	클러스터링을 위한 데이터 세트를 무작위로 생성해 줍니다. 군집 지정 개수에 따라 여러 가지 클러스터링을 위한 데이터 세트를 쉽게 만들어 줍니다.

분류나 회귀 위한 연습용 예제 데이터 구성

- 일반적으로 딕셔너리 형태

딕셔너리 키 구성

키	개별 키가 가리키는 데이터 세트 의미	타입
<code>data</code>	피쳐의 데이터 세트	넘파이 배열(ndarray) 타입
<code>target</code>	분류일 때 : 레이블 값 데이터 세트 회귀일 때 : 숫자 결과값 데이터 세트	넘파이 배열(ndarray) 타입
<code>target_names</code>	개별 레이블 이름	넘파이 배열 or 리스트(list) 타입
<code>feature_names</code>	피쳐 이름	넘파이 배열 or 리스트(list) 타입
<code>DESCR</code>	세트에 대한 설명&각 피쳐의 설명	스트링 타입

4. Model Selection 모듈 소개

(1) 학습/테스트 데이터 세트 분리 - `train_test_split()`

- 학습 데이터 세트로만 학습&예측 → 정확도 100%
⇒ 예측 수행하는 데이터 세트는 학습용 데이터 세트 아닌 전용 테스트 데이터 세트여야 함.

`train_test_split()`

- 원본 데이터 세트에서 학습 및 테스트 데이터 세트 쉽게 분리 가능
- 첫 번째 파라미터 : 피쳐 데이터 세트 / 두 번째 파라미터 : 레이블 데이터 세트

선택적 파라미터 입력받는 목록

<code>test_size</code>	전체 데이터 중 테스트 데이터 세트 샘플링 크기 결정	디폴트 0.25 (25%)
<code>train_size</code>	전체 데이터 중 학습용 데이터 세트 샘플링 크기 결정	
<code>shuffle</code>	데이터 분리 전에 데이터 미리 섞을 지 결정	디폴트 True
<code>random_state</code>	호출할 때마다 동일한 학습/테스트용 데이터 세트 생성 위해 주어지는 난수 값	

- 반환값 튜플 형태
- 순차적으로 학습용 데이터의 피쳐 데이터 세트, 테스트용 데이터의 피쳐 데이터 세트, 학습용 데이터의 레이블 데이터 세트, 테스트용 데이터의 레이블 데이터 세트 반환

(2) 교차 검증

교차 검증

- 목적: 데이터 편중 방지, 과적합 개선
- 여러 학습 데이터 세트 + 검증 데이터 세트 → 반복 학습·평가
- 교차 검증 → 1차 평가 / 테스트 데이터 세트 → 최종 평가
- 활용: 하이퍼 파라미터 튜닝, 모델 최적화

K 폴드 교차 검증

- 가장 보편적인 교차 검증 기법
- 전체 데이터 → **K개 폴드**로 분할
- 매번:
 - K-1개 → 학습 데이터 세트
 - 1개 → 검증 데이터 세트
- K번 반복 → **K개의 평가 결과 평균**
- 사이킷런: `KFold`
- `split()` → 학습용/검증용 인덱스 반환

Stratified K 폴드

- 대상: 불균형한 분포도를 가진 레이블 데이터 집합
- 핵심: 원본 데이터의 레이블 분포 반영
- K 폴드의 레이블 분포 미반영 문제 해결

- 분류(Classification) → 일반적으로 Stratified K 폴드 사용
- 사이킷런: `StratifiedKFold`
- `split()` → 피쳐 데이터 + 레이블 데이터 필요
- 회귀(Regression) → Stratified K 폴드 사용 불가
 - 결정값이 연속된 숫자값이기 때문

교차 검증을 보다 간편하게 — `cross_val_score()`

- 교차 검증 과정 한꺼번에 수행
- 학습 → 예측 → 평가 자동 수행
- 주요 파라미터:
 - `estimator` → 분류/회귀 알고리즘
 - `X` → 피쳐 데이터 세트
 - `y` → 레이블 데이터 세트
 - `scoring` → 평가 지표
 - `cv` → 교차 검증 폴드 수
- 반환값 → 성능 지표 측정값 배열
- Classifier → Stratified K 폴드
- Regressor → K 폴드
- 보통 → 각 평가 결과의 평균 사용
- `cross_validate()` → 여러 평가 지표 + 학습 데이터 성능 + 수행 시간 제공

(3) GridSearchCV-교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

GridSearchCV

- 목적: 최적의 하이퍼 파라미터 도출
- 하이퍼 파라미터 → 알고리즘의 주요 구성 요소, 예측 성능 개선 가능
- 여러 하이퍼 파라미터 값을 순차적으로 적용 → 학습/평가 → 최적의 파라미터 조합 탐색
- 교차 검증 기반으로 최적 하이퍼 파라미터 탐색
- `param_grid`에 기술된 모든 파라미터를 순차적으로 적용
- 단점: 파라미터를 순차적으로 테스트하므로 수행시간이 상대적으로 오래 걸림

수행 횟수

- 파라미터 조합 6개 + `CV=3`
- $6 \times 3 = 18$ 회 학습/평가

주요 파라미터

- `estimator` → classifier, regressor, pipeline
- `param_grid` → 튜닝할 파라미터명 + 여러 파라미터 값
- `scoring` → 예측 성능 평가 방법
- `cv` → 교차 검증 폴드 수
- `refit=True` → 최적 하이퍼 파라미터로 estimator 재학습

주요 결과

- `params` → 적용된 개별 하이퍼 파라미터값
- `rank_test_score` → 하이퍼 파라미터별 성능 순위
 - 1위 → 최적의 하이퍼 파라미터
- `mean_test_score` → CV의 폴딩 테스트 세트에 대한 평균 평가값
- `best_params_` → 최적 하이퍼 파라미터값
- `best_score_` → 최적 하이퍼 파라미터의 최고 평가 결과
- `best_estimator_` → 최적 하이퍼 파라미터로 재학습된 Estimator

전체 과정

- 학습 데이터 → **GridSearchCV**
- 하이퍼 파라미터 조합별 교차 검증
- → 최적 하이퍼 파라미터 탐색
- → 최적 하이퍼 파라미터로 재학습
- → 별도의 테스트 데이터 세트에서 최종 평가

5. 데이터 전처리

(1) 데이터 인코딩

데이터 전처리

- ML 알고리즘 적용 전 기본적인 데이터 처리
- **Garbage In, Garbage Out**
- 결손값(NaN, Null) → 고정된 다른 값으로 변환
 - Null 적음 → 피처의 평균값 등으로 대체
 - Null 대부분 → 해당 피처 드롭
 - 중요 피처의 Null → 업무 로직 등을 검토해 정밀한 대체 값 선정
- 문자열 값 → 숫자형으로 변환
 - 카테고리형 → 인코딩
 - 텍스트형 → 피처 벡터화 또는 삭제
 - 주민번호, 문자열 아이디 등 식별자 피처 → 삭제

레이블 인코딩(Label Encoding)

- 카테고리 피처 → 코드형 숫자 값으로 변환
- 사이킷런 → `LabelEncoder`
- `fit()` + `transform()` → 레이블 인코딩
- `classes_` → 원본값과 인코딩 값 확인
- `inverse_transform()` → 인코딩 값을 다시 디코딩
- 단점 → 숫자의 크고 작음에 대한 특성이 작용할 수 있음
- 선형 회귀와 같은 ML 알고리즘에는 적용하지 않아야 함
- 트리 계열 ML 알고리즘 → 숫자의 특성을 반영하지 않아 레이블 인코딩 사용 가능

원-핫 인코딩(One-Hot Encoding)

- 레이블 인코딩의 문제점을 해결하기 위한 인코딩 방식
- 고유 값에 해당하는 새로운 피처 추가
- 해당 고유 값 → 1
- 나머지 → 0
- 사이킷런 → `OneHotEncoder`
- 입력값 → 2차원 데이터 필요
- 변환 결과 → 희소 행렬(Sparse Matrix)
- `toarray()` → 밀집 행렬(Dense Matrix) 변환
- 판다스 → `get_dummies()`
 - 문자열 카테고리 값을 숫자형으로 변환할 필요 없이 바로 변환 가능

(2) 피처 스케일링과 정규화

- 피처 스케일링: 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업
- 표준화: 평균 0, 분산 1인 값으로 변환
- 정규화: 값을 0~1 범위로 변환
- 벡터 정규화: 개별 벡터의 크기를 맞추기 위해 개별 벡터를 모든 피처 벡터의 크기로 나눔

(3) StandardScaler

- 표준화를 지원하는 클래스
- 개별 피처를 평균 0, 분산 1인 값으로 변환

(4) MinMaxScaler

- 데이터값을 0~1 사이의 범위로 변환
- 음수 값이 있으면 1~1 값으로 변환

(5) 학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

- `fit()` → 변환 기준 정보 설정
- `transform()` → 설정된 정보로 데이터 변환
- `fit_transform()` → `fit()` + `transform()`
- 학습 데이터 → `fit()` + `transform()` 또는 `fit_transform()`
- 테스트 데이터 → 학습 데이터로 fit된 Scaler 객체의 `transform()` 만 사용
- 테스트 데이터에 `fit()` / `fit_transform()` 사용 금지
- 핵심: 학습 데이터와 테스트 데이터에 동일한 스케일링 기준 적용

3장 평가

1. 정확도(Accuracy)

- 실제 데이터에서 예측 데이터가 얼마나 같은지 판단하는 지표

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

- 직관적으로 모델 예측 성능을 나타내는 평가 지표
- 불균형한 레이블 값 분포에서는 성능을 왜곡할 수 있음
- 정확도 하나만으로 성능을 평가하지 않음
- 불균형한 데이터 세트에서는 적합한 평가 지표가 아님

2. 오차 행렬

- 오차 행렬(혼동행렬): 이진 분류의 예측 오류와 오류 유형을 함께 나타내는 지표
- 실제 레이블과 예측 레이블의 조합으로 TN, FP, FN, TP 구성
- TN (True Negative)
 - 예측값: Negative(0)
 - 실제값: Negative(0)
- FP (False Positive)
 - 예측값: Positive(1)
 - 실제값: Negative(0)
- FN (False Negative)
 - 예측값: Negative(0)
 - 실제값: Positive(1)
- TP (True Positive)
 - 예측값: Positive(1)
 - 실제값: Positive(1)
- 앞 문자 T/F
 - True → 예측값과 실제값이 같음
 - False → 예측값과 실제값이 틀림
- 뒤 문자 Negative/Positive
 - Negative(0) / Positive(1) → 예측 결과 값을 의미
- 사이킷런
 - confusion_matrix() → 오차 행렬 계산
- 정확도
 - $\text{Accuracy} = (TN + TP) / (TN + FP + FN + TP)$
 - 오차 행렬의 TN과 TP에 좌우됨
- 불균형한 이진 분류
 - Positive 데이터 건수가 매우 작음
 - Negative로 예측하는 경향이 강해짐

- **TN**이 매우 커지고 **TP**가 매우 작아짐
- 정확도가 높게 나타나도 **Positive**에 대한 예측 성능을 판단하지 못할 수 있음
- 따라서 정확도만으로는 모델 신뢰도가 떨어질 수 있음

3. 정밀도와 재현율

- 정밀도와 재현율: **Positive** 데이터 세트의 예측 성능에 초점을 맞춘 평가 지표
- 정밀도(Precision)
 - $TP / (FP + TP)$
 - **Positive**로 예측한 대상 중 실제 **Positive**인 데이터의 비율
 - **FP**를 낮추는 데 초점
- 재현율(Recall)
 - $TP / (FN + TP)$
 - 실제 **Positive**인 대상 중 예측과 실제 값이 **Positive**로 일치한 데이터의 비율
 - **FN**을 낮추는 데 초점
 - 민감도(Sensitivity), **TPR**(True Positive Rate)
- 재현율이 중요한 경우
 - 실제 **Positive**를 **Negative**로 잘못 판단하면 업무상 큰 영향이 발생하는 경우
- 정밀도가 중요한 경우
 - 실제 **Negative**를 **Positive**로 잘못 판단하면 업무상 큰 영향이 발생하는 경우

(1) 정밀도/재현율 트레이드오프

- *분류 결정 임계값(Threshold)**을 조정해 정밀도 또는 재현율의 수치를 높일 수 있음
- 정밀도와 재현율은 상호 보완적인 평가 지표
- 어느 한쪽을 높이면 다른 하나의 수치는 떨어지기 쉬움
- 임계값 ↓
 - 재현율 ↑
 - 정밀도 ↓
- 임계값 ↑
 - 정밀도 ↑
 - 재현율 ↓
- **predict_proba()**
 - 개별 데이터별 예측 확률 반환
 - 이진 분류 → 첫 번째 칼럼: 클래스 0, 두 번째 칼럼: 클래스 1의 예측 확률
- 일반적인 분류 결정 임계값 = 0.5
- **precision_recall_curve()**
 - 임계값에 따른 정밀도와 재현율 확인

(2) 정밀도와 재현율의 맹점

- 임계값을 변경해 하나의 성능 지표만 극단적으로 높이는 것은 바람직하지 않음
- **정밀도 100%**
 - 확실한 경우만 Positive로 예측 → **FP = 0**
- **재현율 100%**
 - 모든 데이터를 Positive로 예측 → **FN = 0**
- 따라서 정밀도와 재현율 중 하나만 높고 다른 하나가 낮은 분류는 성능이 좋지 않은 분류로 간주할 수 있음
- 가장 좋은 성능 평가 → 정밀도와 재현율 모두 높은 수치

4. F1 스코어

- **F1 스코어: 정밀도와 재현율을 결합한 지표**
- 정밀도와 재현율이 어느 한 쪽으로 치우치지 않을 때 상대적으로 높은 값
- 공식
 - $F1 = 2 \times \text{정밀도} \times \text{재현율} / (\text{정밀도} + \text{재현율})$
- 정밀도와 재현율 중 어느 한쪽이 낮으면 F1 스코어도 낮아짐
- 따라서 정밀도와 재현율을 함께 고려하여 분류 성능을 평가할 수 있음
- 사이킷런
 - `f1_score()` → F1 스코어 계산

5. ROC 곡선과 AUC

- **ROC 곡선**
 - 이진 분류의 예측 성능 측정에 사용
 - *FPR(False Positive Rate)*이 변할 때 *TPR(True Positive Rate)*이 어떻게 변하는지를 나타내는 곡선
 - X축 → **FPR**
 - Y축 → **TPR(재현율)**
- **TPR(재현율)**
 - $TP / (FN + TP)$
 - 민감도(Sensitivity)
- **TNR(특이성)**
 - $TN / (FP + TN)$
 - 특이성(Specificity)
- **FPR**
 - $FP / (FP + TN)$
 - $1 - TNR = 1 - \text{특이성}$
- **ROC 곡선과 임계값**
 - 분류 결정 임계값을 변경하여 FPR을 0~1까지 변화
 - 임계값 1 → FPR 0
 - 임계값 0 → FPR 1

- **AUC(Area Under Curve)**
 - ROC 곡선 밑의 면적
 - 일반적으로 1에 가까울수록 좋은 수치
 - 0.5 → 랜덤 수준의 이진 분류
 - FPR이 작은 상태에서 큰 TPR을 얻을수록 좋은 성능
- 사이킷런
 - `roc_curve()` → FPR, TPR, 임계값 반환
 - `roc_auc_score()` → ROC AUC 값 계산